



琴音袅袅

—— 一个乐器交易租借平台 ——

概要设计规约

指导教师：杜庆峰

小组成员

陈甫彬 2253715

刘子赫 2250694

杨兆镇 2252712

修订历史

编写日期	SEPG	版本	说明	作者
2024.11.18		1.0	根据需求分析规约，完成概要设计内容	陈甫彬、刘子赫、杨兆镇
2024.11.20		1.1	对数据库 E-R 图进行建模分析	陈甫彬、刘子赫、杨兆镇
2024.11.23		1.2	分析 E-R 图，得出简单的接口类图以及分析	陈甫彬、刘子赫、杨兆镇
2024.11.24		1.3	修改已实现的程序，导入界面设计	陈甫彬、刘子赫、杨兆镇
2024.11.25		1.4	最终完善概要设计规约	陈甫彬、刘子赫、杨兆镇

目录

1. 引言	1
1.1 概要设计依据	1
1.2 参考资料	1
1.3 假定和约束	2
1.3.1 假定	2
1.3.2 约束	3
2. 概要设计	5
2.1 系统总体架构设计	5
2.2 系统软件结构设计	6
2.3 接口设计	9
2.3.1 登录注册微服务	9
2.3.2 AI 对话微服务	11
2.3.3 收藏商品微服务	13
2.3.4 评论微服务	15
2.3.5 支付订单管理微服务	17
2.3.6 卖出订单管理微服务	21
2.3.7 支付微服务	23
2.3.8 商品管理微服务	26
2.4 界面设计	29
2.4.1 登录界面	29
2.4.2 注册界面	30
2.4.3 找回密码&修改密码界面	30
2.4.4 商店界面	31
2.4.5 个人信息界面	32
2.4.6 我的收藏界面	33
2.4.7 我的商品界面	33
2.4.8 消费订单界面	33
2.4.9 店铺订单界面	34
2.4.10 上架商品界面	34
2.4.11 乐器大师界面	35

2.4.12 商品界面	35
2.4.13 管理员界面	36
2.4.14 用户管理界面	36
2.4.15 商品管理界面	37
2.4.16 评论管理界面	37
2.5 数据库设计	38
2.5.1 逻辑设计	38
2.5.2 物理设计	39
2.6 系统出错处理设计	42
2.6.1 出错信息	42
2.6.2 补救措施	42
2.6.3 系统维护设计	43

1. 引言

1.1 概要设计依据

- **需求分析规约：**基于上一次的需求分析规约，我们小组这次进行了进一步的概要设计。
- **用户调研和问卷结果：**在项目初期，我们通过问卷调查收集了用户的反馈，了解了他们在使用乐器交易和租借功能时的期望。这也是我们进行概要设计的基础。
- **技术选型：**在选择技术栈时，我们考虑了项目的需求和特性，前端使用 Vue.js、后端使用 Node.js 或 Python Flask，数据库选择了 MySQL。这些技术选型帮助我们确定了系统架构和设计方向。
- **行业实践：**我们小组参考了当前 Web 应用开发的最佳实践，比如响应式设计、数据隐私保护和支付安全等，以确保系统设计符合现代标准。
- **课程内容：**结合老师上课讲的要点以及给出的模板，我们小组确定了这次需要进行的工作：进行包图等体系设计，进行 E-R 图的设计，E-R 中的数据表设计，以及简洁的接口类图的设计，初步的界面设计。

1.2 参考资料

- **需求分析规约**

标题：乐器交易平台需求分析规约

文件编号：20241116-NA-001

发表日期：2024 年 11 月 16 日

出版单位：课程项目组

来源：小组内部文档

- **用户调研问卷**

标题：乐器交易平台用户调研与问卷分析

文件编号：20241020-UR-001

发表日期：2024 年 10 月 20 日

出版单位：软件工程课程小组

来源：小组内部文档

- **系统设计标准**

标题：Web 应用系统设计与开发标准

文件编号：WASDS-2023

发表日期：2023 年 6 月 1 日

出版单位：国际软件工程协会（ISEA）

来源：<https://www.isea.org/>

• 技术文档

标题：Vue.js 官方文档

文件编号：VJS-2023

发表日期：2023 年 3 月 10 日

出版单位：Vue.js 官方团队

来源：<https://vuejs.org/>

1.3 假定和约束

1.3.1 假定

• 用户基础：

- 平台的主要用户包括买家、卖家和管理员，所有用户在访问平台时需先注册并登录。
- 用户熟悉基本的网络操作，能够在平台上完成商品查询、交易或租赁操作。
- 管理员假定为平台内部工作人员，主要进行维护平台秩序，协助处理不法行为，同时处理纠纷等工作。
- 卖家用户主要假定为贩卖乐器或者出租二手乐器的商家。
- 买家用户根据需求调研，假定为音乐爱好者、初学音乐尝试者、以及音乐上手者。

• 交易行为：

- 交易行为基于用户信任机制，平台提供商品描述、图片和评价等信息供参考，但不负责实体商品质量保障。
- 平台会向买家和卖家提供支持，如交易纠纷处理、租赁时限提醒等。

- **租赁行为:**

- 租赁默认以“按日计费”模式进行，特殊租赁需求（如按小时或长期租赁）由买卖双方自行协商。

- 租赁的乐器需要卖家给与押金金额，并提供取货与归还的方式说明。
 - 计费从买家收到货后开始，至以买家邮递归还确认后结束
 - 对于乐器途中的丢失、损坏问题，卖家有联系管理员的途径，以及走法律途径。

- **管理员行为:**

- 管理员可以管理用户权限（如禁用违规用户账户）、下架违规商品，以及对平台评价进行监控。

- 管理员有权查看并审核大众交易的记录，确保合规性。
 - 管理员可以帮助买家、卖家处理交易中的问题，或者走法律途径

- **数据存储更新:**

- 平台所有商品、交易、租赁记录、用户信息和评论会存储在数据库中，并实时更新。

- **支付结算:**

- 支付使用第三方支付接口（如支付宝、微信支付、PayPal 等），平台仅提供支付通道，无需保存用户支付信息。

- 交易完成后，卖家可提现资金，提现周期假定为 2~5 个工作日。

1.3.2 约束

- **技术约束:**

- 平台为 Web 端开发，需兼容主流浏览器（如 Chrome、Firefox、Safari）。
 - 使用指定的技术栈（如前端 Vue.js，后端 Spring Boot，数据库 MySQL）开发，且需要在项目周期内完成。

- 集成支付接口，但不允许保存用户的敏感支付信息，如银行卡号或 CVV。

- **性能约束:**

- 平台需要支持至少 500 并发用户的同时访问，且响应时间需控制在 2 秒内。

- 页面加载速度要求小于 3 秒，并支持延迟加载图片和商品数据以优化性能。

- **法律与政策约束：**

- 符合中国的电子商务法律法规（如消费者权益保护法、隐私政策等）。
- 用户数据存储需符合 GDPR 或其他隐私保护法案的要求。
- 不允许平台交易非法商品（如赃物、盗版乐谱等）。

- **用户约束：**

- 所有用户需要通过邮箱或手机号码进行实名认证，确保交易的真实性。
- 买家和卖家需签署平台提供的电子协议，承诺遵守交易和租赁规则。

- **开发约束：**

- 根据课上老师要求以及其他课程的约束，开发周期限制在 1 个月半左右完成，包括需求分析、建模、开发、测试和部署。
- 受学生身份限制，项目预算有限，需优先使用开源工具和框架，避免额外的高额授权费用。

- **安全约束：**

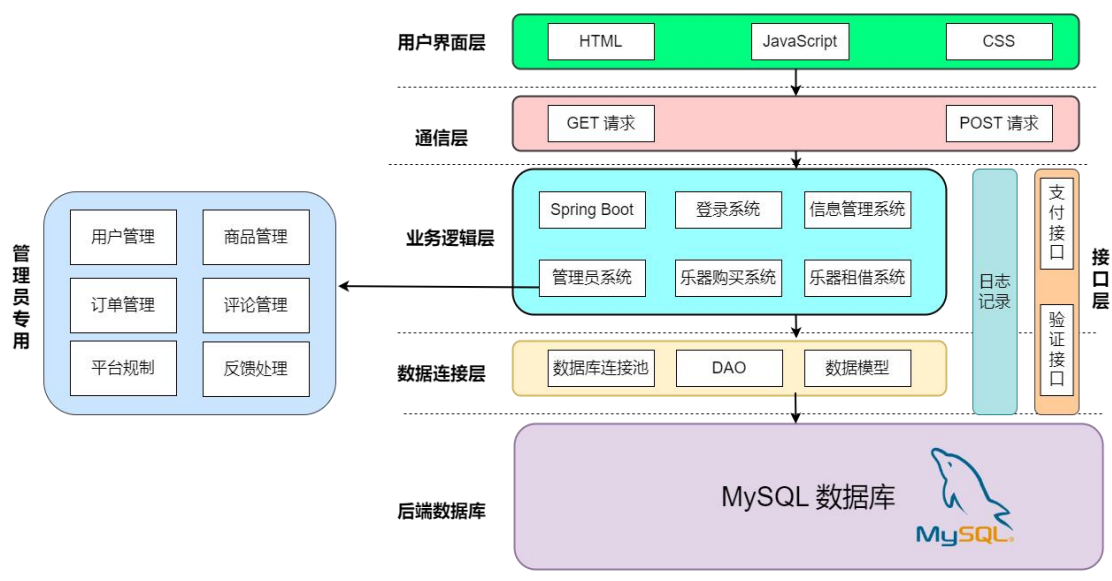
- 平台必须实现 HTTPS 协议，确保用户数据的传输安全。
- 需要实现防止 SQL 注入、跨站脚本攻击（XSS）和跨站请求伪造（CSRF）的安全机制。

- **可维护约束：**

- 平台代码需遵守团队制定的代码规范，提供详细注释。
- 代码在 Github 进行版本管理。
- 系统需设计成模块化架构，符合设计模式便于后期扩展或修复漏洞。

2. 概要设计

2.1 系统总体架构设计



本系统采用分层架构设计，分为以下几个主要部分：用户界面层（UI 层）、通信层、业务逻辑层、接口层、数据访问层和后端数据库层。系统以高效性、可扩展性和用户友好性为设计目标，结合现代主流技术实现。

· 用户界面层（UI 层）

用户界面层负责用户与系统之间的交互，采用了 HTML、CSS 和 JavaScript 作为核心技术。HTML 提供了页面结构，通过语义化的标签组织界面内容，确保良好的可访问性和可维护性。CSS 实现了界面的美观布局，包括响应式设计，支持多种设备访问。JavaScript 提供动态交互功能，例如用户验证、动态数据更新和实时反馈，提升了用户体验。

· 通信层

作为用户界面层与业务逻辑层之间的桥梁，采用 GET 和 POST 请求实现前后端的数据交互。GET 请求用于从服务器获取数据，例如用户信息加载、商品列表查询等。POST 请求用于向服务器提交数据，例如用户登录信息、订单提交等操作。通信层在数据传递中确保了请求的安全性和高效性，并与后端的 Spring Boot 紧密结合，利用其内置的 HTTP 请求处理机制，支持异步操作和数据格式的灵活处理，实现快速响应和稳定性能。

· 业务逻辑层

业务逻辑层使用 Spring Boot 框架构建。Spring Boot 是基于 Java 的现代化微服务框架，具有快速开发、高效运行的特性，支持自动配置和嵌入式服务器，简化了开发流程。

主要功能模块包括

- 登录系统：实现用户的认证与授权，保证系统的安全性。
- 信息管理系统：提供用户信息和数据的增删查改功能。
- 管理员系统：管理员可以管理乐器商品、租借记录及用户反馈。
- 乐器购买和租借系统：支持乐器的在线购买和租借功能，提供详细的商品信息和灵活的交易模式。

▪ 接口层

接口层连接外部服务和内部系统，支付接口集成第三方支付服务，支持多种支付方式，保障交易的安全性和便捷性。验证接口实现用户身份的第三方认证，提高了系统的安全性。日志记录通过日志追踪系统运行状态，记录错误信息，为后续的调试和优化提供支持。

▪ 数据访问层

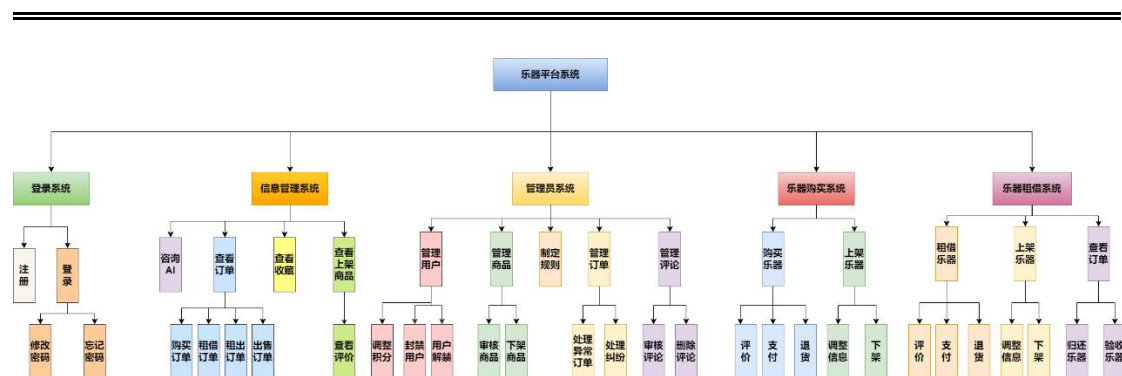
数据访问层负责数据的存储和处理逻辑，采用 DAO 模式 以隔离数据访问逻辑，确保代码结构清晰、易维护。数据库连接池提供高效的连接管理机制，提高了系统处理高并发请求的能力。数据模型对复杂数据进行了结构化建模，方便数据的存储、查询和更新操作。

▪ 后端数据库层

系统使用 MySQL 数据库作为后端存储。MySQL 是一种高性能的关系型数据库管理系统，支持复杂的查询语句和事务管理，保障了数据的一致性和完整性。系统采用了规范化设计和索引优化策略，提升了数据查询的效率。数据库结构设计覆盖用户信息、商品信息、订单记录、租借记录等，提供完整的数据支持。

2.2 系统软件结构设计

• 子系统结构图



基于之前的需求分析规约，我们的乐器平台系统大致分为 5 个子系统：登录系统、信息管理系统、管理员系统、乐器购买系统、乐器租借系统。

登陆系统提供注册、登录操作，其中登录还包含安全验证以及找回密码的操作，来确保登录安全性以及以防用户忘记密码。

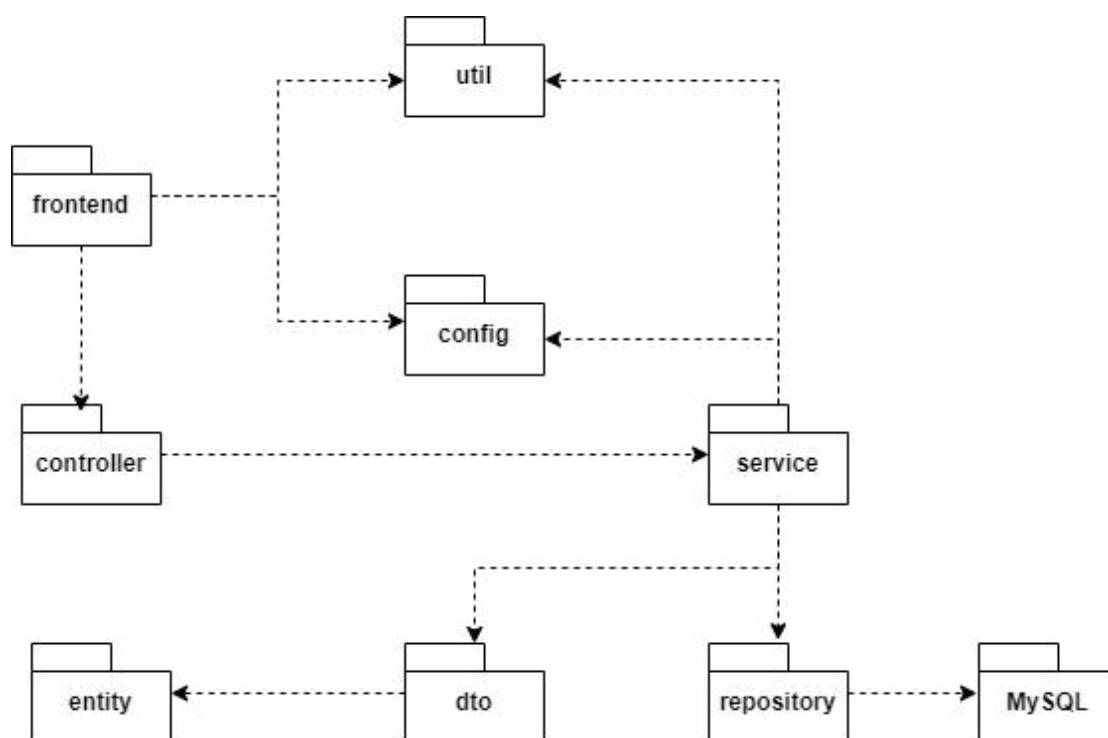
信息管理系统提供查看订单、查看收藏、查看上架商品。查看订单功能中，用户可以查看自己购买的订单、租借的订单、租出的订单、出售的订单。查看收藏用于用户标记喜欢的商品，方便以后快速查看。查看上架商品其中提供了评论功能，让商家查看自己商品的评论来调整商品或者营销策略。同时，我们准备接入生成式 AI，让其帮助用户进行乐器的选择等。

管理员系统只有管理员有权限使用。管理员可以管理用户来封禁或者解封用户。管理员可以管理商品的上架审核以及下架不法商品。管理员可以制定平台规则，管理还可以管理用户之间的订单来处理异常订单，处理买卖纠纷。管理员可以审核评论、删除不合理评论。

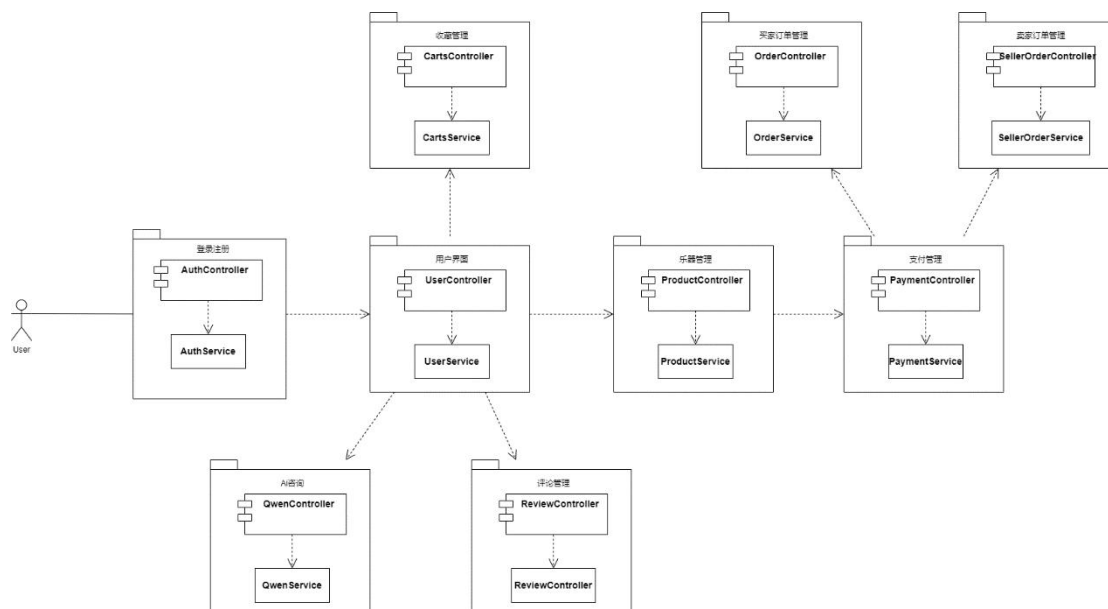
乐器购买系统用于卖家、买家之间的乐器交易。买家可以在购买乐器的时候，进行支付，评价以及退货操作。卖家可以在上架商品后进行商品信息介绍的调整、商品的下架操作。

乐器租借系统用于处理卖家、买家之间的租借流程。买家在租借的功能中同样可以进行评价、支付、退货操作。卖家可以在上架过程中进行信息调整、商品下架。卖家和买家可以共同追踪订单情况，确保乐器的归还以及验收。

• 体系结构包图



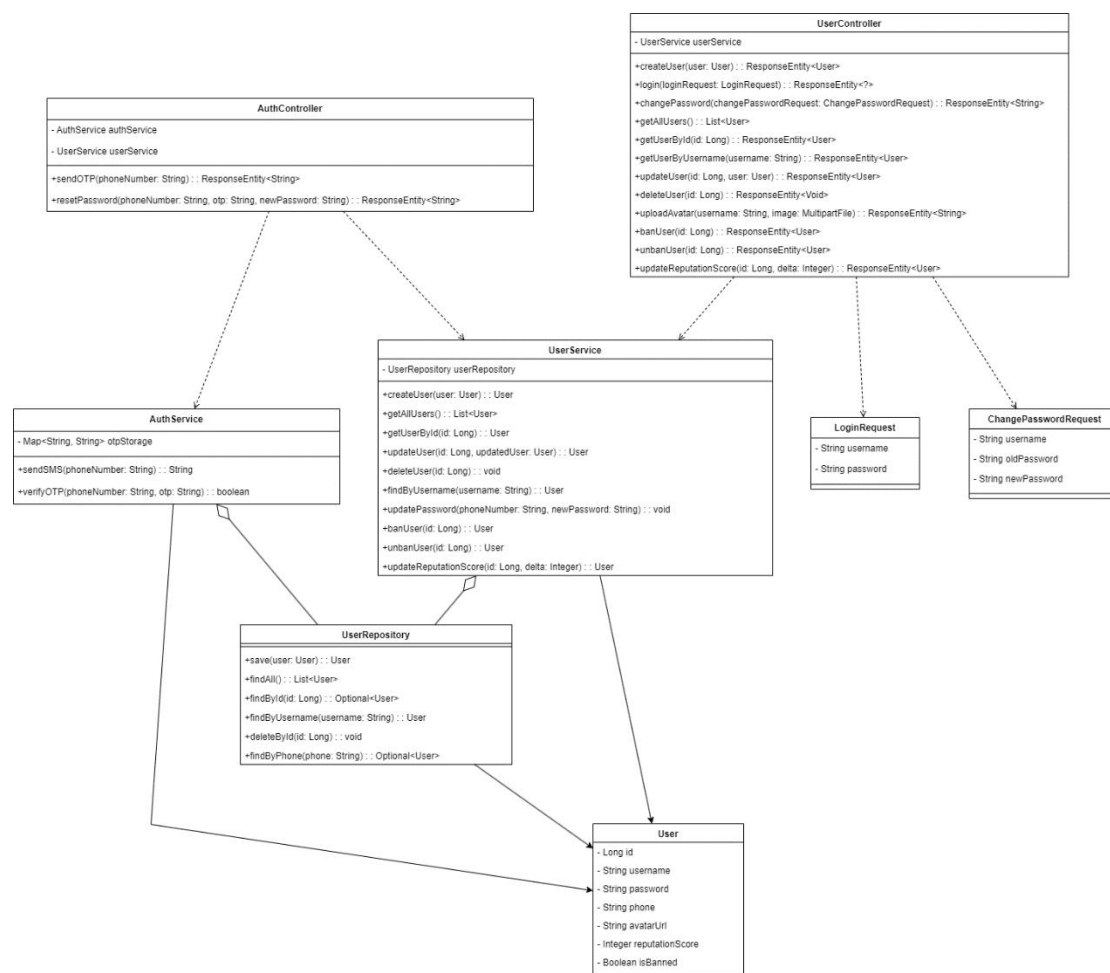
• 软件架构设计



以上软件结构设计基于 java Spring boot，架构简单明了

2.3 接口设计

2.3.1 登录注册微服务



• Controller

1.AuthController

类功能概述

AuthController 负责用户身份验证的接口，包括发送 OTP 验证码和重置密码。

类方法签名

ResponseEntity<String> sendOTP(String phoneNumber);

ResponseEntity<String> resetPassword(String phoneNumber, String otp, String newPassword);

2.UserController

类功能概述

UserController 提供用户管理相关的接口，如用户创建、登录、密码修改、信息查询、用户状态管理等功能。

类方法签名

```
ResponseEntity<User> createUser(User user);
ResponseEntity<?> login(LoginRequest loginRequest);
ResponseEntity<String> changePassword(ChangePasswordRequest
changePasswordRequest);
List<User> getAllUsers();
ResponseEntity<User> getUserById(Long id);
ResponseEntity<User> getUserByUsername(String username);
ResponseEntity<Void> deleteUser(Long id);
ResponseEntity<User> updateAvatar(Long id, MultipartFile image);
ResponseEntity<User> banUser(Long id);
ResponseEntity<User> unbanUser(Long id);
ResponseEntity<User> updateReputationScore(Long id, Integer delta);
```

• Service

1.AuthService

类功能概述

AuthService 提供身份验证的具体服务实现，包括发送短信验证码和验证 OTP。

类方法签名

```
String sendSMS(String phoneNumber);
boolean verifyOTP(String phoneNumber, String otp);
```

2.UserService

类功能概述

UserService 实现用户管理的核心逻辑，包括用户的创建、更新、删除、查询等操作，以及用户信誉分和密码管理。

类方法签名

```
User createUser(User user);
List<User> getAllUsers();
User getUserById(Long id);
User updateUser(Long id, User updatedUser);
void deleteUser(Long id);
User findByUsername(String username);
void updatePassword(String phoneNumber, String newPassword);
User banUser(Long id);
User unbanUser(Long id);
User updateReputationScore(Long id, Integer delta);
```

- Repository

1. UserRepository

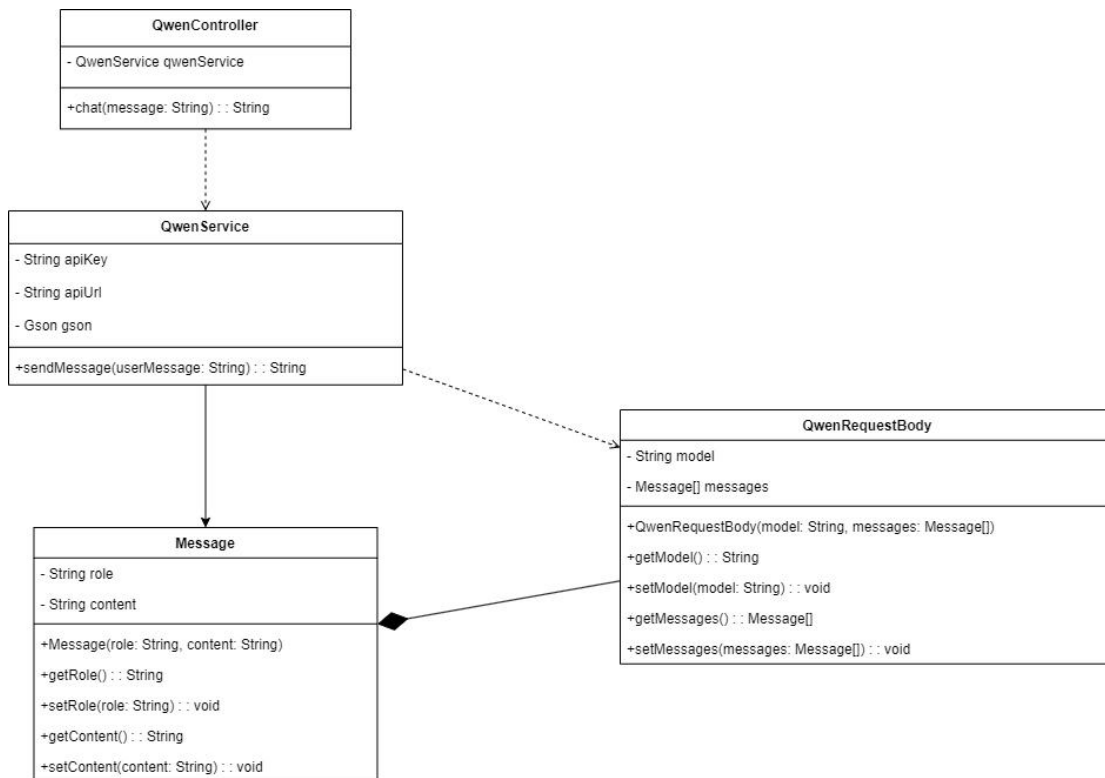
类功能概述

UserRepository 负责与数据库的交互，提供用户数据的存储、查询和删除等基础操作。

类方法签名

```
User save(User user);
List<User> findAll();
Optional<User> findById(Long id);
Optional<User> findByUsername(String username);
Optional<User> findByPhone(String phone);
void deleteById(Long id);boolean verifyOTP(String phoneNumber, String otp);
```

2.3.2 AI 对话微服务



- **Controller**

- 1. **QwenController**

- 类功能概述

- QwenController 负责处理用户向 AI 模型发送对话消息的请求。

- 类方法签名

- String chat(String message);

- **Service**

- 1. **QwenService**

- 类功能概述

- QwenService 实现与外部 AI 模型的交互逻辑，包括处理用户消息并调用 AI 接口返回响应。

- 类方法签名

- String sendMessage(String userMessage);

- **Others**

- 1. **QwenRequestBody**

类功能概述

QwenRequestBody 封装了与 AI 模型交互时的请求数据, 包括模型名称和消息内容。

类方法签名

```
QwenRequestBody(String model, Message[] messages);  
String getModel();  
void setModel(String model);  
Message[] getMessages();  
void setMessages(Message[] messages);List<User> findAll();
```

2.Message

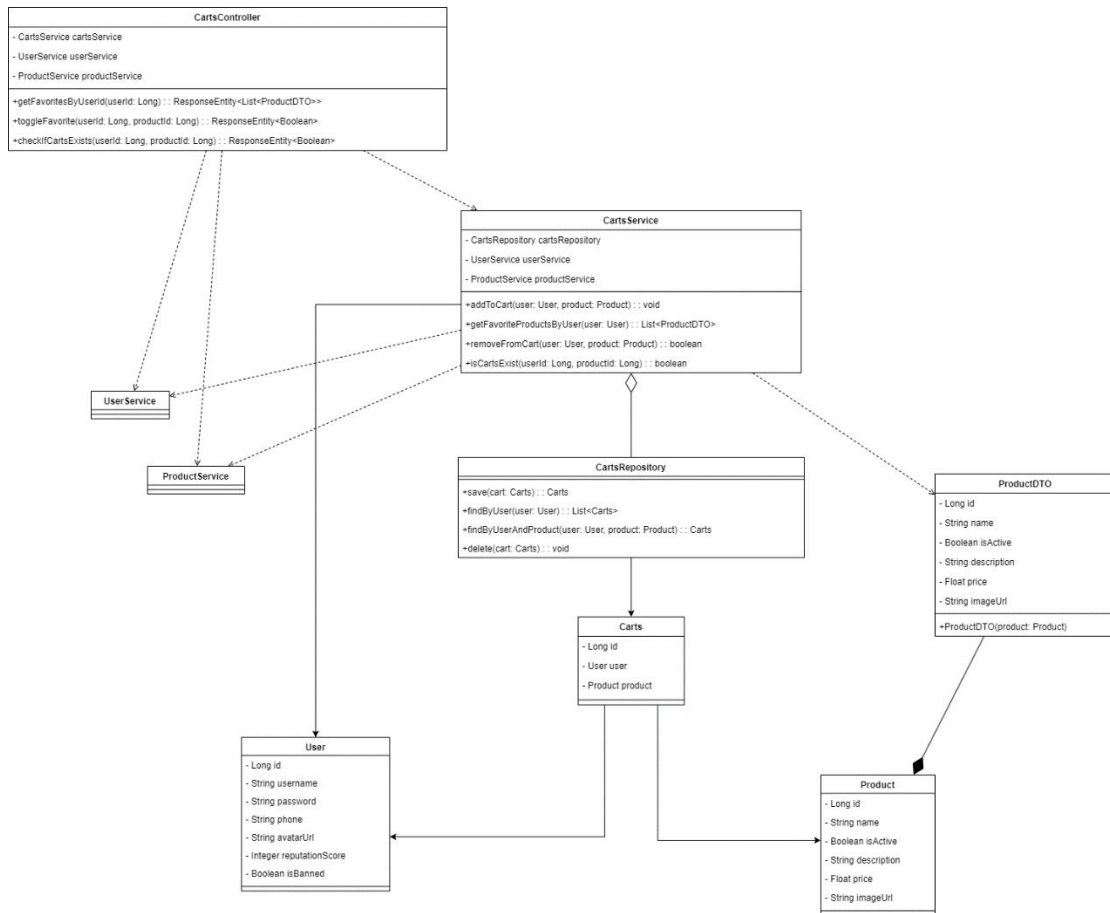
类功能概述

Message 类封装了单个对话消息的内容, 包括消息的角色(用户或 AI) 和消息文本。

类方法签名

```
Message(String role, String content);  
String getRole();  
void setRole(String role);  
String getContent();  
void setContent(String content);
```

2.3.3 收藏商品微服务



• Controller

1.CartsController

类功能概述

CartsController 提供与用户收藏夹相关的接口，例如添加收藏、移除收藏、获取用户收藏列表等。

类方法签名

`ResponseEntity<List<ProductDTO>> getFavoritesByUserId(Long userId);`

`ResponseEntity<Boolean> toggleFavorite(Long userId, Long productId);`

`ResponseEntity<Boolean> checkIfCartExists(Long userId, Long productId);`

• Service

1.CartsService

类功能概述

CartsService 实现收藏功能的核心逻辑，例如添加收藏、移除收藏、查询用户收藏等操作。

类方法签名

```
void addToCart(User user, Product product);  
List<ProductDTO> getFavoriteProductsByUser(User user);  
boolean removeFromCart(User user, Product product);  
boolean isCartExist(Long userId, Long productId);
```

• Repository

1.CartsRepository

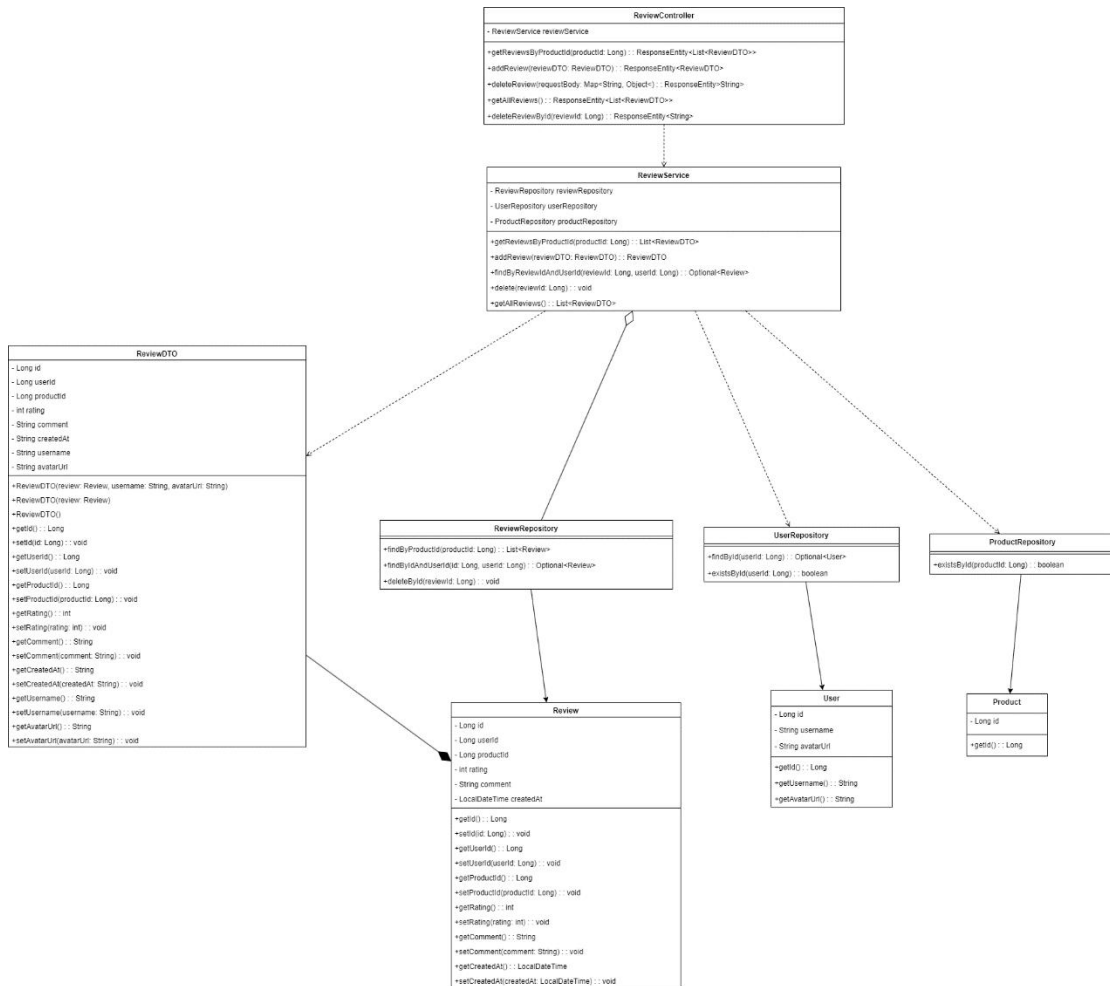
类功能概述

CartsRepository 负责与数据库交互，提供收藏数据的存储、查询和删除等基础操作。

类方法签名

```
Carts save(Carts cart);  
List<Carts> findByUser(User user);  
Carts findByUserAndProduct(User user, Product product);  
void delete(Carts cart);
```

2.3.4 评论微服务



• Controller

1.ReviewController

类功能概述

ReviewController 提供评论相关的接口，例如添加评论、删除评论、查询商品评论等。

类方法签名

`ResponseEntity<List<ReviewDTO>> getReviewsByProductId(Long productId);`

`ResponseEntity<ReviewDTO> addReview(ReviewDTO reviewDTO);`

`ResponseEntity<Void> deleteReview(Long reviewId);`

• Service

1.ReviewService

类功能概述

ReviewService 实现评论功能的核心逻辑，例如评论的添加、删除和查询。

类方法签名

```
List<ReviewDTO> getReviewsByProductId(Long productId);  
ReviewDTO addReview(ReviewDTO reviewDTO);  
void deleteReview(Long reviewId);
```

• Repository

1. ReviewRepository

类功能概述

ReviewRepository 负责与数据库交互，提供评论数据的存储、查询和删除等基础操作。

类方法签名

```
List<Review> findByProductId(Long productId);  
Optional<Review> findById(Long reviewId);  
void deleteById(Long reviewId);
```

• Others

1. ReviewDTO

类功能概述

ReviewDTO 用于封装评论数据的传输对象，包含评论的关键信息。

类方法签名

```
ReviewDTO(Review review);  
Long getId();  
Long getUserId();  
Long getProductId();  
Integer getRating();  
String getComment();  
String getCreateAt();
```

2.3.5 支付订单管理微服务



• Controller

1.OrderController

类功能概述

OrderController 提供订单管理的接口，例如获取订单列表、查询订单详情、确认订单和退货处理等。

类方法签名

```

ResponseEntity<List<Order>> getAllOrders();
ResponseEntity<Order> getOrderBy(Long id);

```

```
ResponseEntity<List<Map<String, Object>>> getOrdersByUserAndFilters(Long
userId, String orderType, String orderStatus);
ResponseEntity<Order> confirmOrder(Long orderId);
ResponseEntity<Order> returnOrder(Long orderId);
```

- Service

1. OrderService

类功能概述

OrderService 实现订单管理的核心逻辑，包括获取订单、更新订单状态、根据用户和过滤条件查询订单等。

类方法签名

```
List<Order> getAllOrders();
Order getOrderById(Long id);
Order updateOrderStatus(Long id, String status);
List<Map<String, Object>> getOrdersByUserAndFilters(Long userId, String
orderType, String orderStatus);
Order confirmOrder(Long orderId);
Order returnOrder(Long orderId);
```

- Repository

1. OrderRepository

类功能概述

OrderRepository 负责与数据库交互，提供订单数据的存储、查询和删除等基础操作。

类方法签名

```
List<Order> findByUserId(Long userId);
Optional<Order> findById(Long id);
```

2. PurchaseOrderRepository

类功能概述

PurchaseOrderRepository 负责与数据库交互，处理购买订单的数据存储和查询。

类方法签名

只需 Java Spring Boot 封装好的方法即可。

3. RentalOrderRepository

类功能概述

RentalOrderRepository 负责与数据库交互，处理租赁订单的数据存储和查询。

类方法签名

只需 Java Spring Boot 封装好的方法即可。

4. ProductRepository

类功能概述

ProductRepository 负责与数据库交互，处理产品的数据存储和查询。

类方法签名

`boolean existsById(Long productId);`

• Others

1.OrderDTO

类功能概述

OrderDTO 用于封装订单数据的传输对象，包含订单的关键信息，如用户 ID、产品 ID、数量、总价、地址和订单状态。

类方法签名

`OrderDTO(Long id, Long userId, Long productId, int quantity, BigDecimal totalPrice, String address, LocalDateTime createAt, OrderStatus orderStatus);`

`Long getId();`

`Long getUserId();`

`Long getProductId();`

`int getQuantity();`

`BigDecimal getTotalPrice();`

`String getAddress();`

`LocalDateTime getCreateAt();`

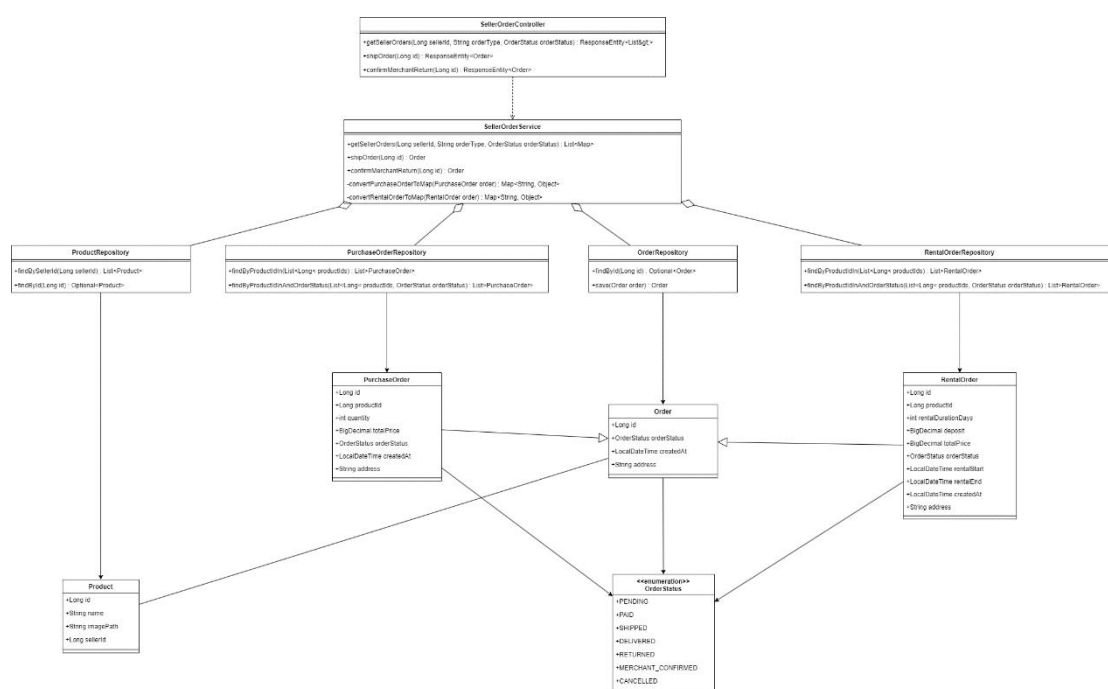
`OrderStatus getOrderStatus();`

```

void setId(Long id);
void setUserId(Long userId);
void setProductId(Long productId);
void setQuantity(int quantity);
void setTotalPrice(BigDecimal totalPrice);
void setAddress(String address);
void setCreateAt(LocalDate createAt);
void setOrderStatus(OrderStatus orderStatus);

```

2.3.6 卖出订单管理微服务



• Controller

1. SellerOrderController

类功能概述

SellerOrderController 提供卖家订单管理的接口，例如获取卖家订单、发货、确认退货等操作。

类方法签名

```

ResponseEntity<List<Map<String, Object>>> getSellerOrders(Long sellerId, String orderType, OrderStatus orderStatus);

```

```

ResponseEntity<Order> shipOrder(Long id);

```

```
ResponseEntity<Order> confirmMerchantReturn(Long id);
```

- Service

- 1. SellerOrderService

- 类功能概述

- SellerOrderService 实现卖家订单管理的核心逻辑，包括获取卖家订单、发货操作、确认退货等功能。

- 类方法签名

- ```
List<Map<String, Object>> getSellerOrders(Long sellerId, String orderType,
OrderStatus orderStatus);
Order shipOrder(Long id);
Order confirmMerchantReturn(Long id);
Map<String, Object> convertPurchaseOrderToMap(PurchaseOrder order);
Map<String, Object> convertRentalOrderToMap(RentalOrder order);
```

- Repository

- 1. PurchaseOrderRepository

- 类功能概述

- PurchaseOrderRepository 负责与数据库交互，处理购买订单的数据存储和查询。

- 类方法签名

- ```
List<PurchaseOrder> findByProductIn(List<Long> productIds);
List<PurchaseOrder> findByProductInAndOrderStatus(List<Long> productIds,
OrderStatus orderStatus);
```

- 2. RentalOrderRepository

- 类功能概述

- RentalOrderRepository 负责与数据库交互，处理租赁订单的数据存储和查询。

- 类方法签名

- ```
List<RentalOrder> findByProductIn(List<Long> productIds);
List<RentalOrder> findByProductInAndOrderStatus(List<Long> productIds,
OrderStatus orderStatus);
```

---

### 3. OrderRepository

#### 类功能概述

OrderRepository 负责与数据库交互，提供订单数据的存储和查询操作。

#### 类方法签名

Optional<Order> findById(Long id);

Order save(Order order);

### 4. ProductRepository

#### 类功能概述

ProductRepository 负责与数据库交互，处理产品的数据存储和查询。

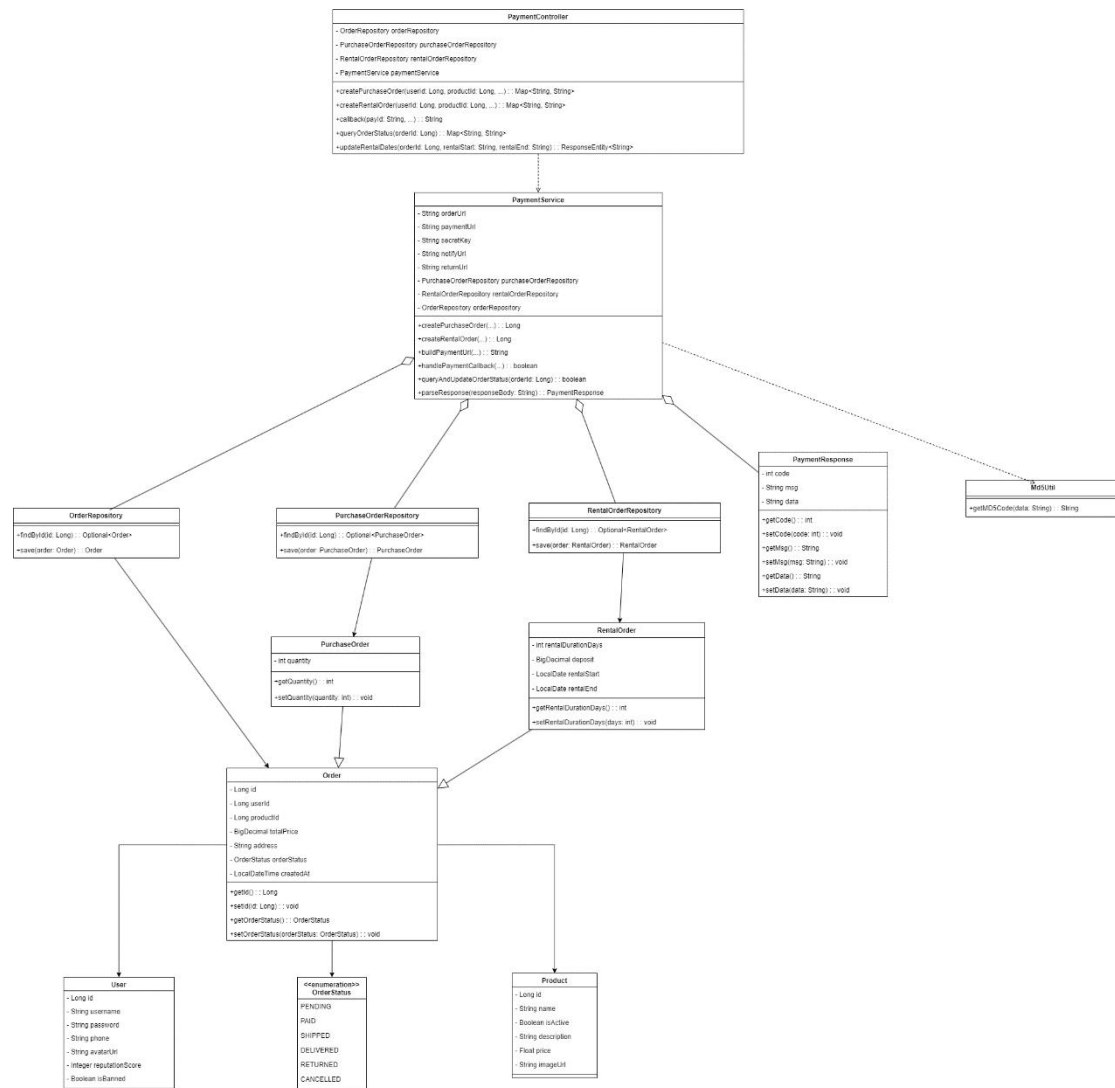
#### 类方法签名

List<Product> findBySellerId(Long sellerId);

Optional<Product> findById(Long id);

### 2.3.7 支付微服务





## • Controller

### 1. PaymentController

#### 类功能概述

PaymentController 提供支付管理的接口，例如创建支付订单、取消支付订单、处理租赁支付等操作。

#### 类方法签名

Map<String, String> createPurchasePayment(Long userId, Long productId, int quantity);

Map<String, String> createRentalPayment(Long userId, Long productId, int rentalDurationDays);

ResponseEntity<String> cancelPurchasePayment(Long orderId);

ResponseEntity<String> cancelRentalPayment(Long rentalId);



---

```
ResponseEntity<String> updateRentalPaymentStatus(Long rentalId, String
newStatus);
```

- Service

## 1. PaymentService

### 类功能概述

PaymentService 实现支付管理的核心逻辑, 包括创建购买支付、创建租赁支付、取消支付订单、更新支付状态等功能。

### 类方法签名

```
Map<String, String> createPurchasePayment(Long userId, Long productId, int
quantity);
```

```
Map<String, String> createRentalPayment(Long userId, Long productId, int
rentalDurationDays);
```

```
boolean cancelPurchasePayment(Long orderId);
```

```
boolean cancelRentalPayment(Long rentalId);
```

```
boolean updateRentalPaymentStatus(Long rentalId, String newStatus);
```

```
PaymentResponse processPayment(String paymentMethod, String paymentData);
```

- Repository

## 1. OrderRepository

### 类功能概述

OrderRepository 负责与数据库交互, 提供订单数据的存储、查询和删除等基础操作。

### 类方法签名

```
Optional<Order> findById(Long id);
```

```
Order save(Order order);
```

## 2. PurchaseOrderRepository

### 类功能概述

PurchaseOrderRepository 负责与数据库交互, 处理购买订单的数据存储和查询。

### 类方法签名

---

```
Optional<PurchaseOrder> findById(Long id);
PurchaseOrder save(PurchaseOrder purchaseOrder);
```

### 3. RentalOrderRepository

#### 类功能概述

RentalOrderRepository 负责与数据库交互，处理租赁订单的数据存储和查询。

#### 类方法签名

```
Optional<RentalOrder> findById(Long id);
RentalOrder save(RentalOrder rentalOrder);
```

- Others

### 1. PaymentResponse

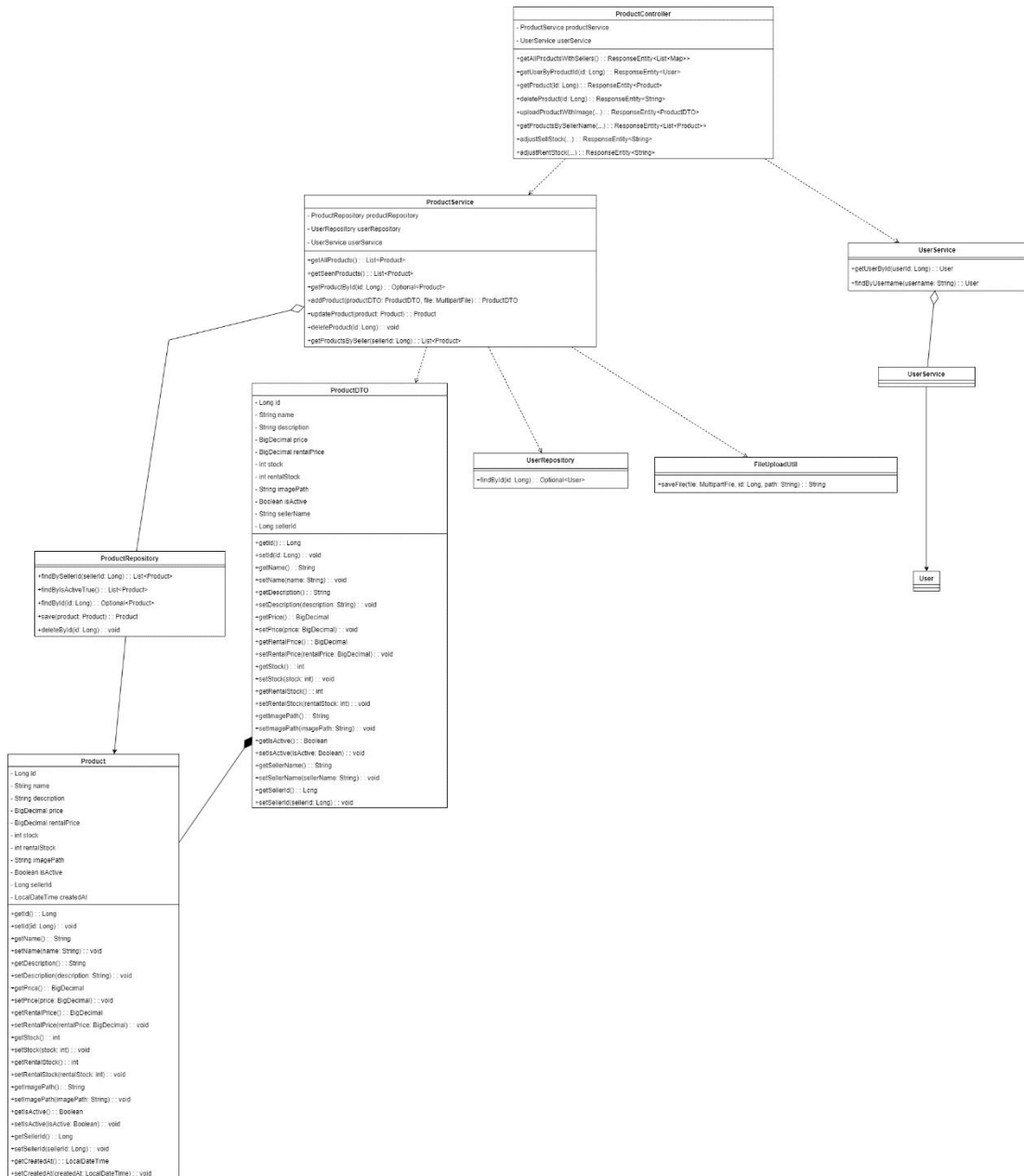
#### 类功能概述

PaymentResponse 用于封装支付响应数据的传输对象，包含支付的状态、交易ID、支付方式等信息。

#### 类方法签名

```
String getCode();
void setCode(String code);
String getTransactionId();
void setTransactionId(String transactionId);
String getStatus();
void setStatus(String status);
String getPaymentMethod();
void setPaymentMethod(String paymentMethod);
String getData();
void setData(String data);
```

### 2.3.8 商品管理微服务



## • Controller

### 1. ProductController

#### 类功能概述

ProductController 提供商品管理的接口，包括获取商品列表、查询商品详情、上传商品图片、添加和删除商品等操作。

#### 类方法签名

`ResponseEntity<List<Map<String, Object>>> getProductsForSeller(Long sellerId);`

`ResponseEntity<ProductDTO> getProductDetails(Long productId);`

---

```
ResponseEntity<String> uploadProductImage(Long productId, MultipartFile image);
ResponseEntity<ProductDTO> addProduct(ProductDTO productDTO);
ResponseEntity<String> updateProduct(Long productId, ProductDTO productDTO);
ResponseEntity<String> deleteProduct(Long productId);
```

- Service

- 1. ProductService

- 类功能概述

- ProductService 实现商品管理的核心逻辑，包括获取卖家的商品列表、查询商品详情、添加商品、更新商品和删除商品等功能。

- 类方法签名

- List<Product> getProductsForSeller(Long sellerId);
    - Product getProductDetails(Long productId);
    - ProductDTO addProduct(ProductDTO productDTO);
    - ProductDTO updateProduct(Long productId, ProductDTO productDTO);
    - void deleteProduct(Long productId);
    - Order confirmOrder(Long orderId);
    - Order returnOrder(Long orderId);

- Repository

- 1. ProductRepository

- 类功能概述

- ProductRepository 负责与数据库交互，提供商品数据的存储和查询操作。

- 类方法签名

- List<Product> findBySellerId(Long sellerId);
    - Optional<Product> findById(Long id);
    - Product save(Product product);
    - void deleteById(Long id);

- Others

- 1. ProductDTO

---

## 类功能概述

ProductDTO 用于封装商品数据的传输对象，包含商品的关键信息，如名称、描述、价格、库存、图片路径等。

## 类方法签名

ProductDTO(Long id, String name, String description, BigDecimal price, int stock, String imagePath, boolean isActive, Long sellerId);

```
Long getId();
String getName();
String getDescription();
BigDecimal getPrice();
int getStock();
String getImagePath();
boolean getIsActive();
Long getSellerId();
void setId(Long id);
void setName(String name);
void setDescription(String description);
void setPrice(BigDecimal price);
void setStock(int stock);
void setImagePath(String imagePath);
void setIsActive(boolean isActive);
void setSellerId(Long sellerId);
```

## 2. FileUploadUtil

### 类功能概述

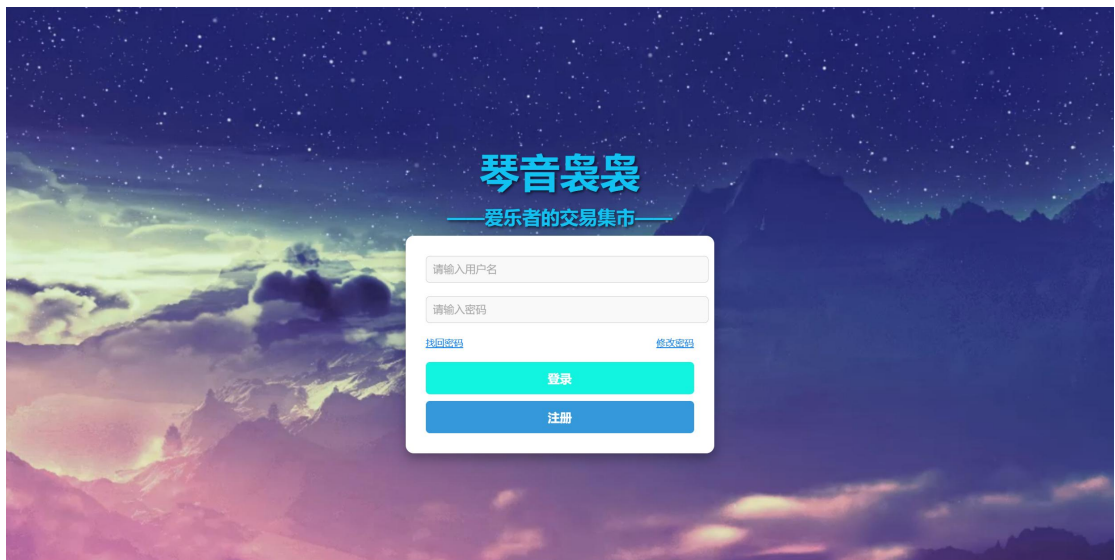
FileUploadUtil 提供文件上传的工具方法，用于将图片文件保存到指定路径。

### 类方法签名

FileUploadUtil 提供文件上传的工具方法，用于将图片文件保存到指定路径。

## 2.4 界面设计

### 2.4.1 登录界面



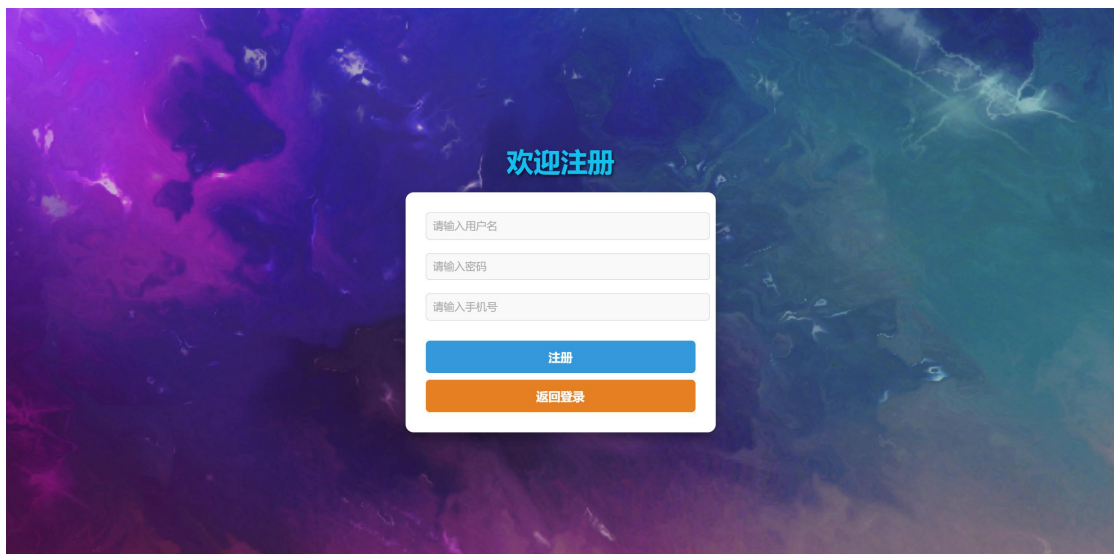
用户进入该界面：

输入正确的用户名和密码，点击登录即可登录。

提供了注册按钮，引导用户进行注册。

提供了修改密码和找回密码选项，引导用户进行密码修改和找回。

## 2.4.2 注册界面

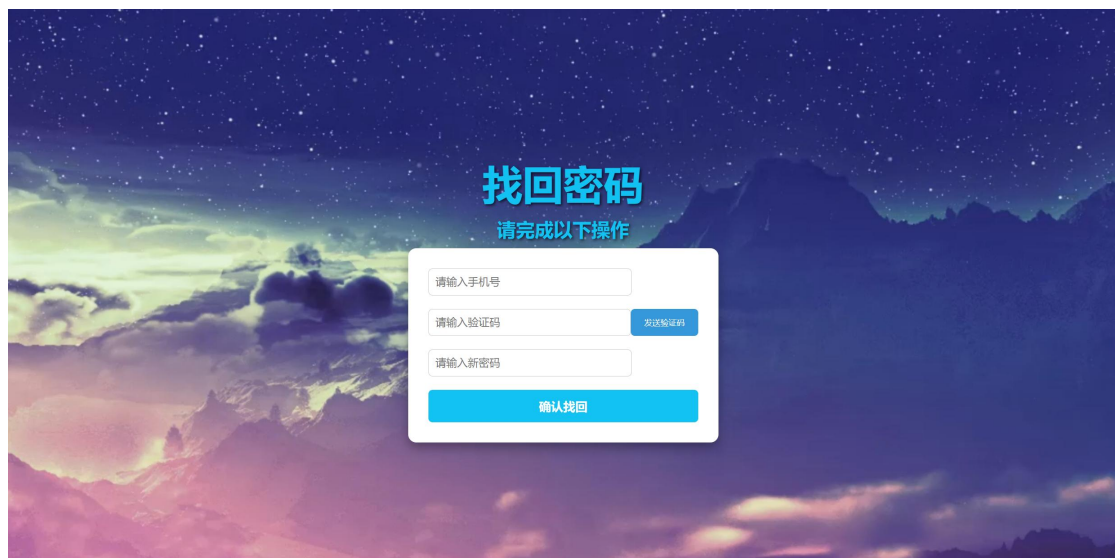


用户进入该界面：

输入唯一的用户名，自己的密码，正确唯一的手机号即可正确注册。

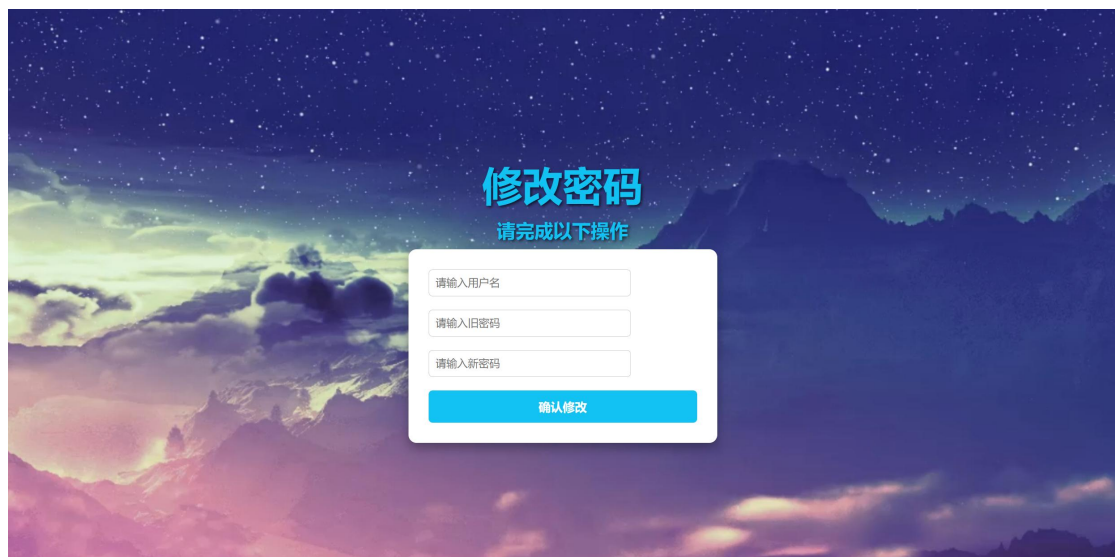
## 2.4.3 找回密码&修改密码界面





用户进入该界面：

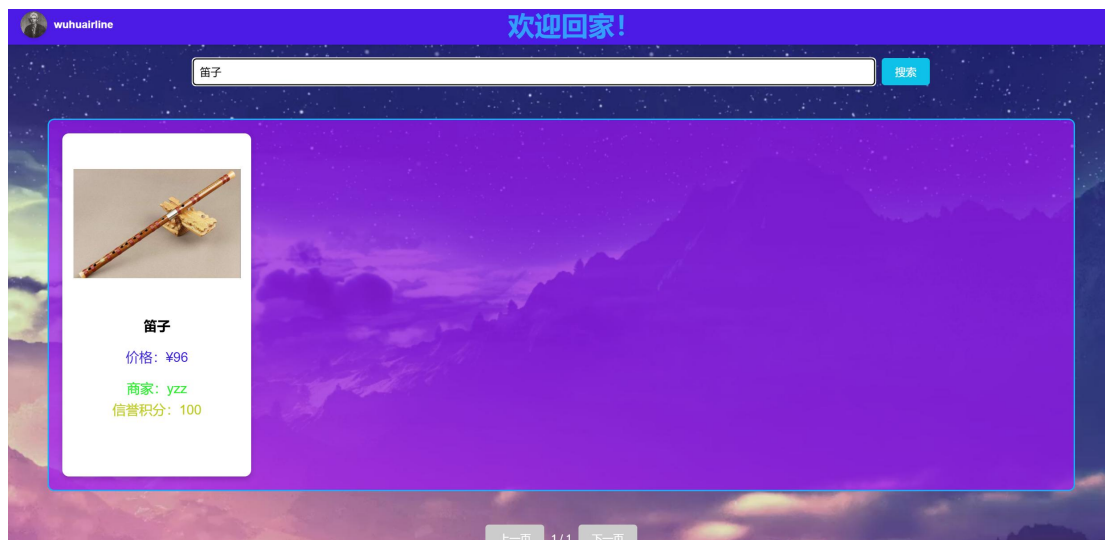
输入自己的手机号以及获取验证码，输入新密码，对应正确即可重置密码。



用户进入该界面：

输入自己的旧密码和新密码，对用正确即可修改密码。

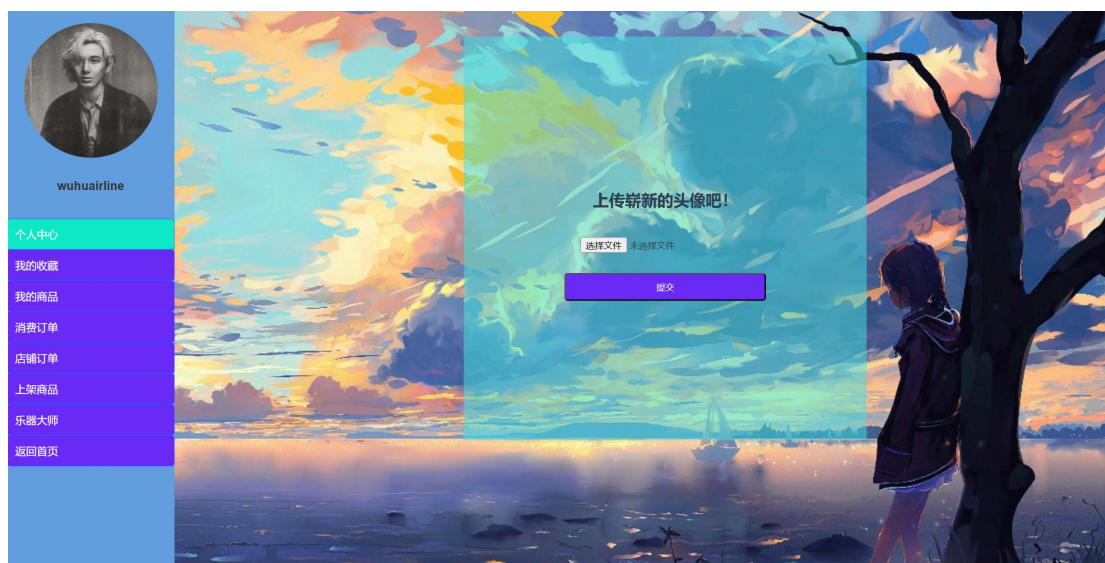
#### 2.4.4 商店界面



用户进入该界面：

可以查看到所有的商品，也可以通过搜索框进行检索。点击上方自己的页面即可进入个人界面。点击对对应的商品进入对应的界面。

## 2.4.5 个人信息界面



用户进入该界面：

可以看到许多选项卡：

个人中心——修改个人信息

我的收藏——查看个人收藏商品

消费订单——查看作为买家买入的所有购买以及租借订单

店铺订单——查看作为卖家出售以及租出的订单

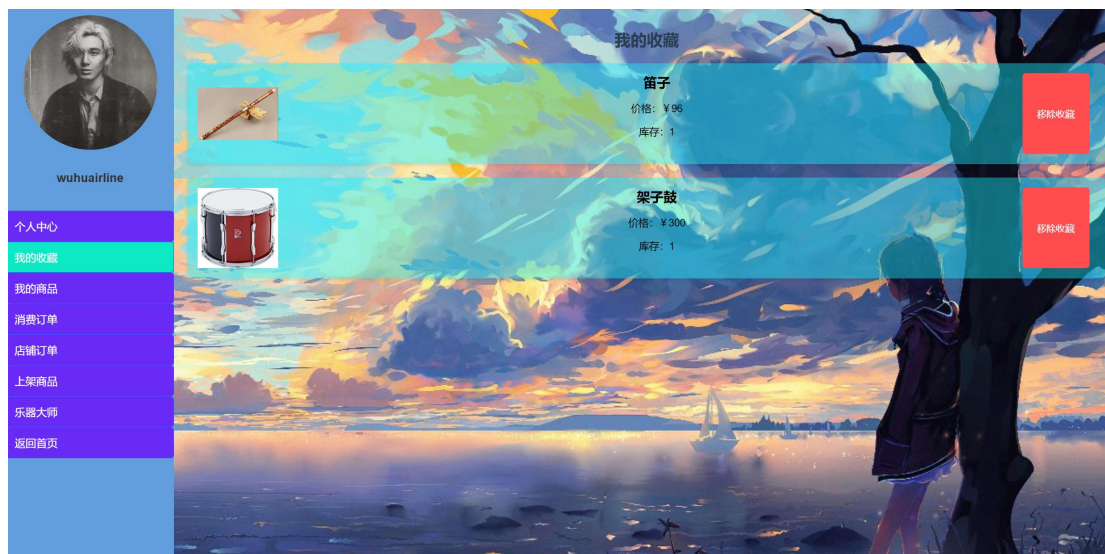
上架商品——上架一个商品的功能

乐器大师——可以与 AI 进行交互

返回首页——回到商店界面



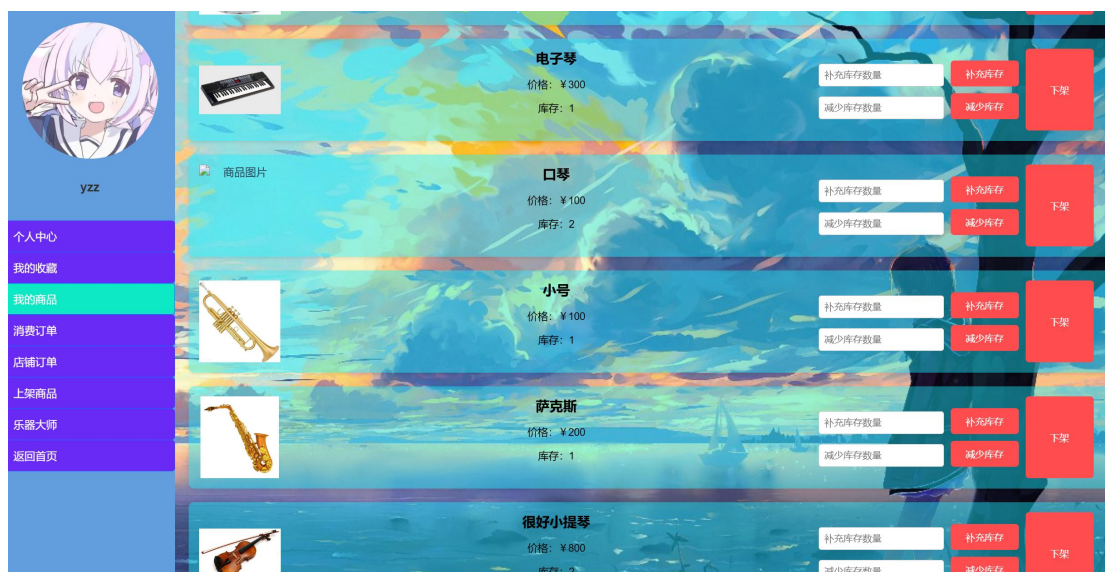
## 2.4.6 我的收藏界面



用户进入该界面：

用户可以查看自己收藏的商品，也可以移除收藏。

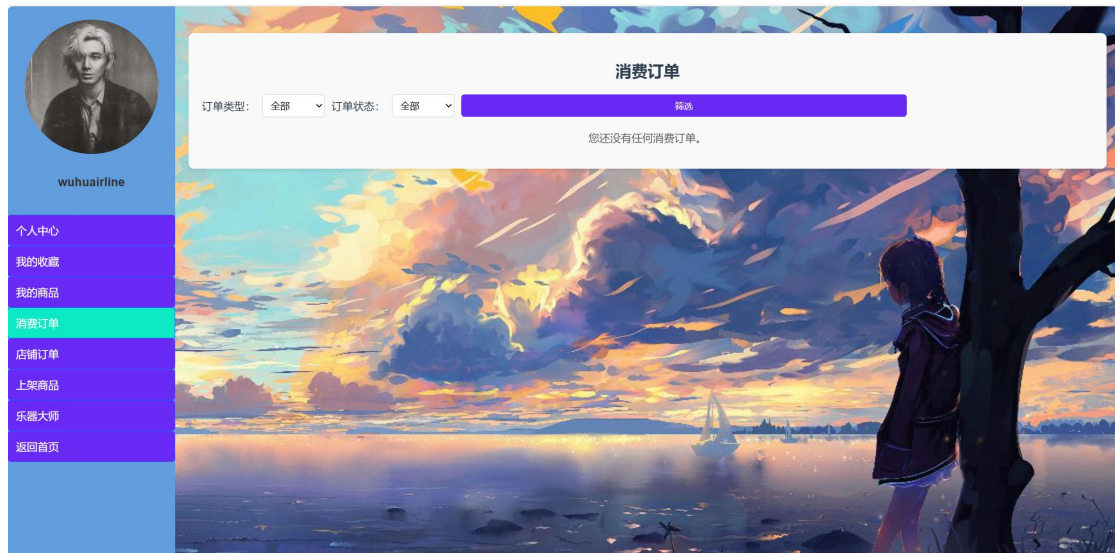
## 2.4.7 我的商品界面



用户进入该界面：

用户可以查看自己上架的商品，可以进库存的补充，以及商品的下架操作

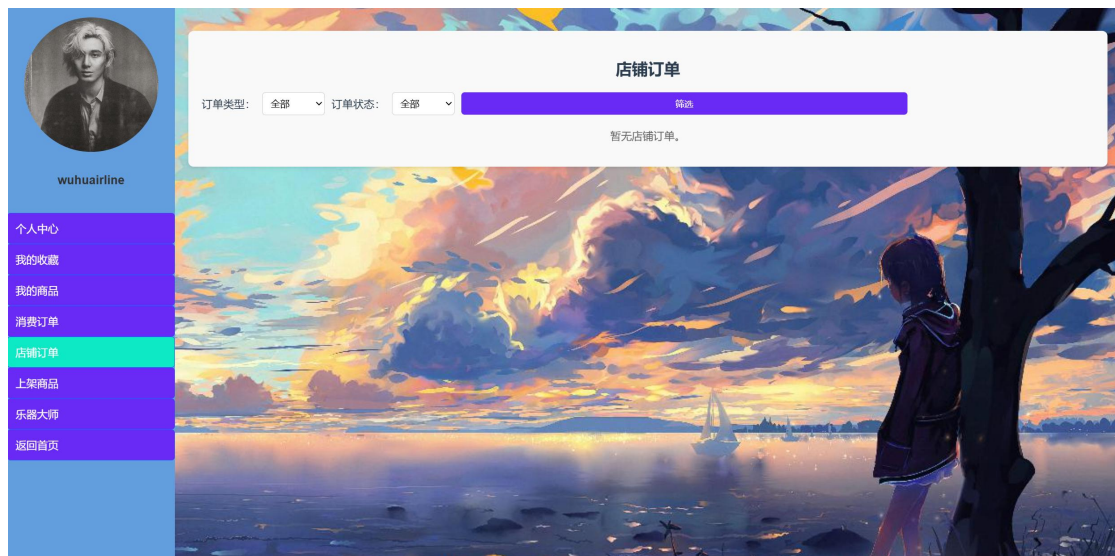
## 2.4.8 消费订单界面



用户进入该界面：

用户可以查看自己作为买家的订单。

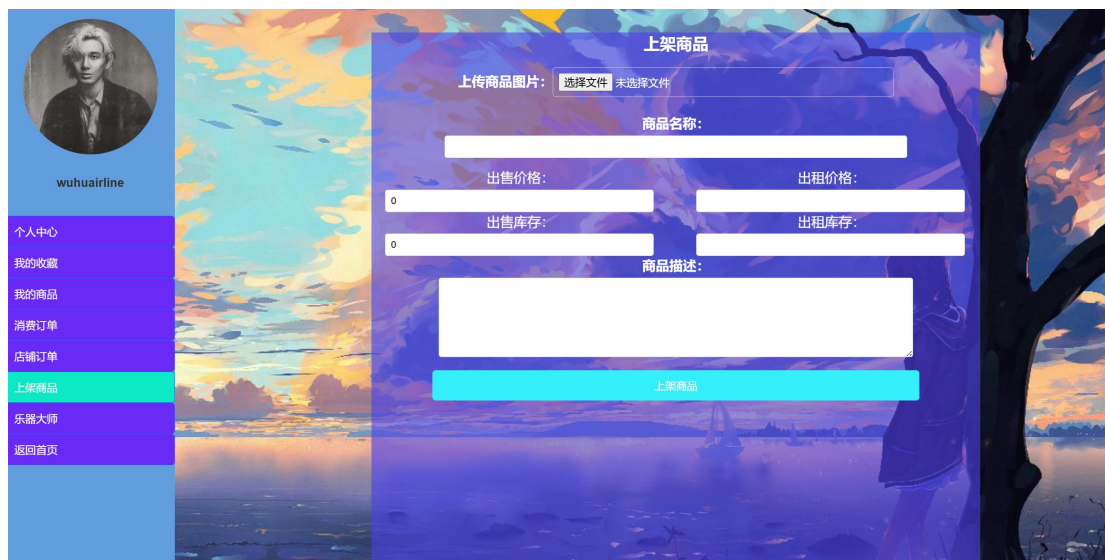
## 2.4.9 店铺订单界面



用户进入该界面：

用户可以查看自己作为卖家的订单。

## 2.4.10 上架商品界面



用户进入该界面：

填写相应的商品信息，上传商品图片即可上架商品。

### 2.4.11 乐器大师界面

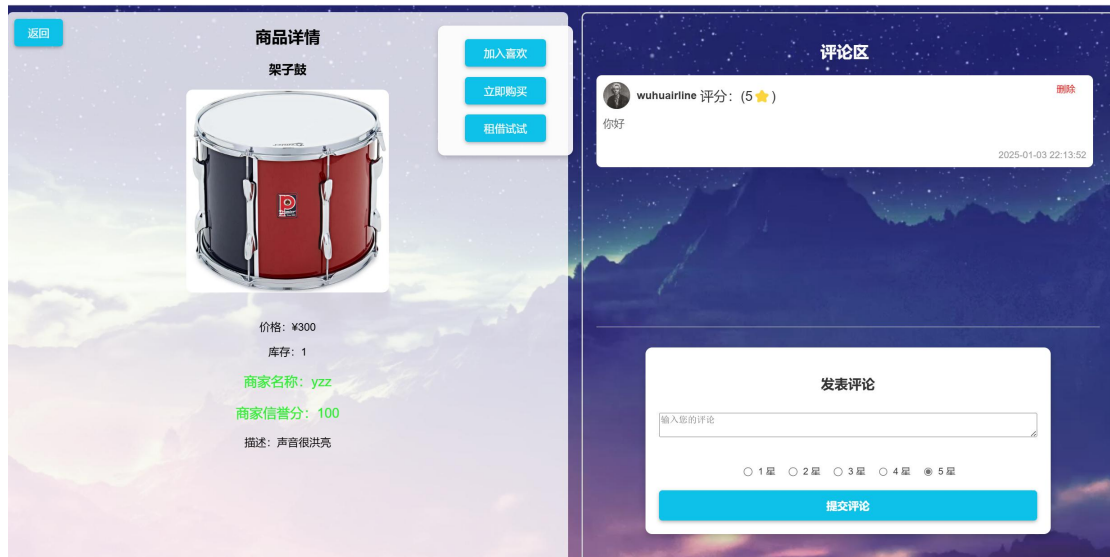


用户进入该界面：

类似 GPT 的界面，用户可以和 AI 进行对话

### 2.4.12 商品界面





用户进入该界面：

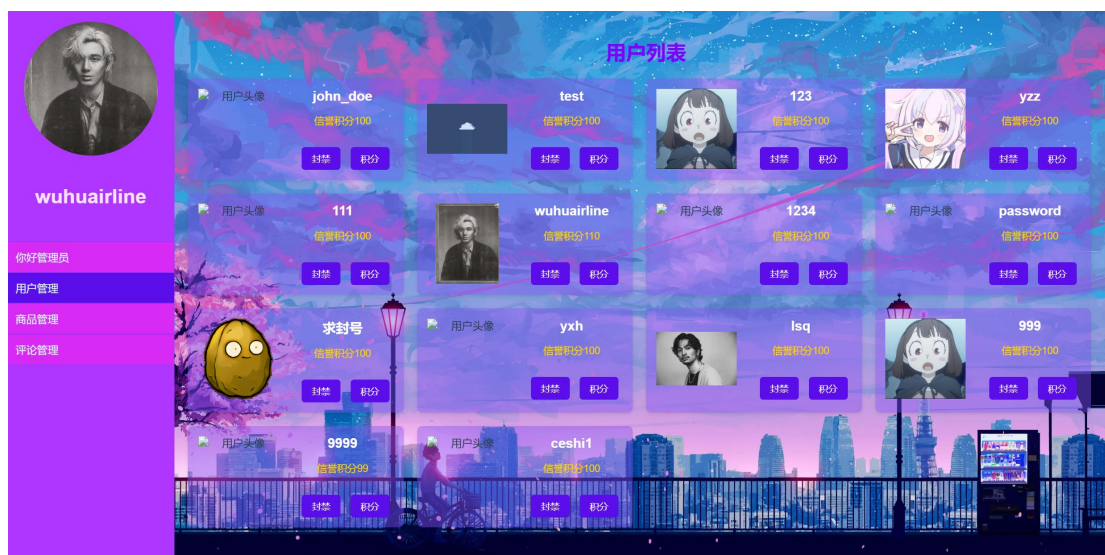
可以查看商品的信息，进行购买、租借、收藏、评论等操作

## 2.4.13 管理员界面



只有管理员登录才能进入管理员界面

## 2.4.14 用户管理界面



管理员进入该界面：

可以进行用户的封禁、解禁，以及积分调整。

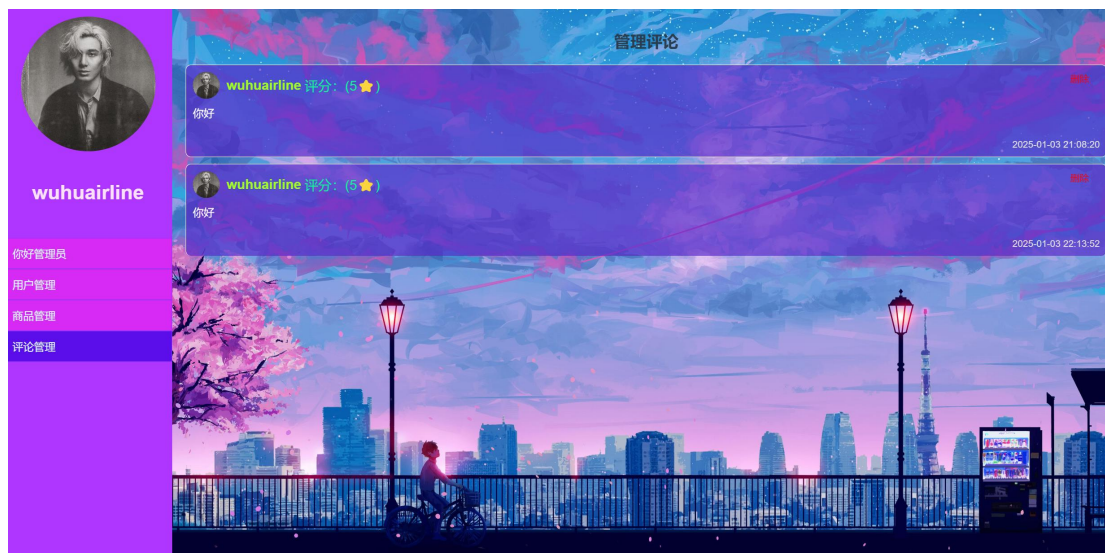
## 2.4.15 商品管理界面



管理员进入该界面：

可以查看所有的商品，点击对应商品条目会进行页面跳转查看商品详细信息。  
点击下架后，即可下架对应商品。

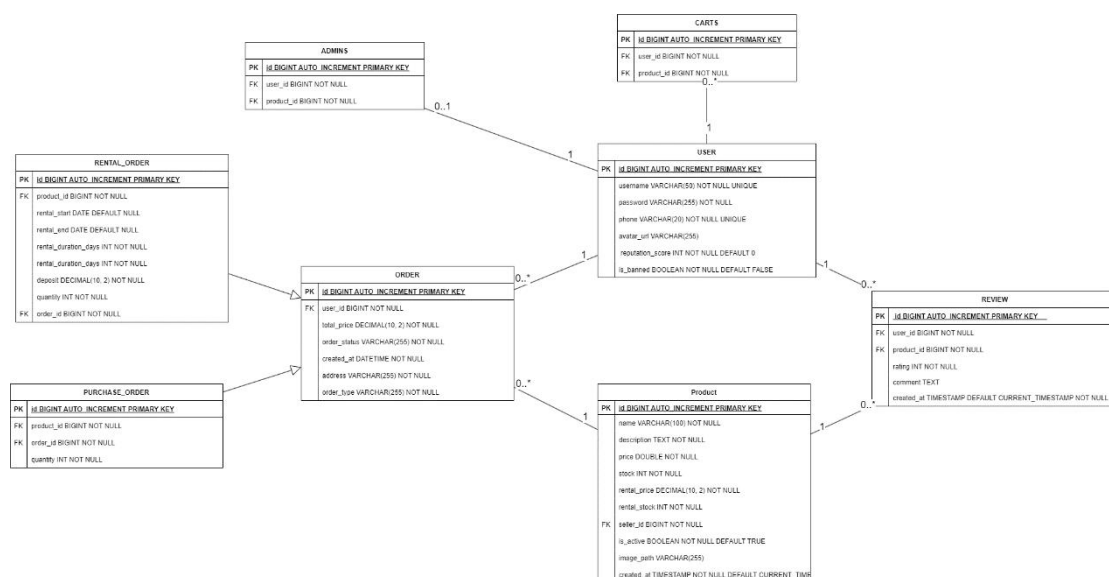
## 2.4.16 评论管理界面



管理员进入该界面：  
可以进行不好评论的删除。

## 2.5 数据库设计

### 2.5.1 逻辑设计



数据库设计的目标：

- 提供用户、乐器、订单、支付、评价、收藏等关键功能的数据存储和管理。
- 支持买家与卖家的主要业务流程，包括购买和租赁乐器。
- 提供平台管理员操作的支持，如商品下架管理，评论管理等。

数据表关系概述如下：



- 用户表（USER）：用于存储用户的基本信息，包括用户名、密码、联系方式。
- 产品表（Product）：存储乐器的详细信息，包括名称、价格、库存和关联的卖家。
- 订单表（ORDER）：存储乐器的交易或租赁订单信息，包括订单类型、状态、金额等。
- 租借订单表（RENTAL\_ORDER）：继承订单父类，显示租借特殊信息。
- 购买订单表（PURCHASE\_ORDER）：继承订单父类，显示购买特殊信息。
- 评价表（REVIEW）：存储用户对乐器的评价信息。
- 购物车表（CARTS）：存储用户收藏的乐器信息。
- 管理员表（ADMINS）：记录哪些用户是内定管理员。

### 2.5.2 物理设计

users 表：

| 字段名              | 数据类型       | 长度  | 说明      | 备注         |
|------------------|------------|-----|---------|------------|
| id               | BIGINT     | —   | 用户主键 ID | 主键，非空，自动递增 |
| username         | VARCHAR    | 50  | 用户名     | 非空         |
| password         | VARCHAR    | 255 | 用户密码    | 非空         |
| phone            | VARCHAR    | 20  | 联系电话    | 可空         |
| avatar-url       | VARCHAR    | 255 | 用户头像路径  | 可空         |
| Reputation-score | INT        | —   | 信誉积分    | 默认值 100    |
| is_banned        | TINYINT(1) | —   | 是否被禁用   | 默认值 0（未禁用） |

products 表：

| 字段名         | 数据类型    | 长度  | 说明      | 备注         |
|-------------|---------|-----|---------|------------|
| id          | BIGINT  | —   | 产品主键 ID | 主键，非空，自动递增 |
| name        | VARCHAR | 100 | 产品名称    | 非空         |
| description | TEXT    | —   | 产品描述    | 可空         |
| price       | —       | 255 | 产品价格    | 非空         |



|              |                |     |        |                  |
|--------------|----------------|-----|--------|------------------|
| stock        | INT            | -   | 库存数量   | 非空，默认值为 0        |
| seller_id    | BIGINT         | -   | 卖家 ID  | 外键，关联 users.id   |
| is_active    | TINYINT(1)     | -   | 产品是否上架 | 默认值 1（1=上架，0=下架） |
| created_at   | TIMESTAMP      | -   | 创建时间   | 默认值为当前时间         |
| image_path   | VARCHAR        | 255 | 产品图片路径 | 可空               |
| rental_price | DECIMAL(10, 2) | -   | 产品租赁价格 | 可空，适用于租赁商品       |
| rental_stock | INT            | -   | 租赁库存数量 | 可空，默认值为 0        |

orders 表:

| 字段名          | 数据类型           | 长度  | 说明      | 备注             |
|--------------|----------------|-----|---------|----------------|
| id           | BIGINT         | -   | 订单主键 ID | 主键，非空，自动递增     |
| user_id      | BIGINT         | -   | 上架用户 ID | 外键，关联 users.id |
| total_price  | DECIMAL(10, 2) | -   | 订单总金额   | 非空             |
| order_status | VARCHAR        | 50  | 订单状态    | 非空             |
| order_type   | VARCHAR        | 31  | 订单类型    | 非空             |
| created_at   | DATETIME       | -   | 创建时间    | 默认值为当前时间       |
| address      | VARCHAR        | 255 | 收货地址    | 非空             |

purchase\_orders 表:

| 字段名        | 数据类型   | 长度 | 说明        | 备注                |
|------------|--------|----|-----------|-------------------|
| id         | BIGINT | -  | 采购订单主键 ID | 主键，非空             |
| product_id | BIGINT | -  | 产品 ID     | 外键，关联 products.id |
| quantity   | INT    | -  | 采购数量      | 非空，默认值为 1         |

rental\_orders 表:

| 字段名 | 数据类型   | 长度 | 说明        | 备注    |
|-----|--------|----|-----------|-------|
| id  | BIGINT | -  | 租借订单主键 ID | 主键，非空 |

|                      |               |   |         |                   |
|----------------------|---------------|---|---------|-------------------|
| product_id           | BIGINT        | - | 产品 ID   | 外键，关联 products.id |
| rental_start         | DATE          | - | 租赁开始日期  | 非空                |
| rental_end           | DATE          | - | 租赁结束日期  | 非空                |
| rental_duration_days | INT           | - | 租赁时长（天） | 非空                |
| deposit              | DECIMAL(10,2) | - | 租赁押金    | 非空，默认值为 0.00      |
| quantity             | INT           | - | 租赁产品数量  | 非空，默认值为 1         |

**carts 表:**

| 字段名        | 数据类型   | 长度 | 说明       | 备注                |
|------------|--------|----|----------|-------------------|
| id         | BIGINT | -  | 购物车主键 ID | 主键，非空，自动递增        |
| user_id    | BIGINT | -  | 用户 ID    | 外键，关联 users.id    |
| product_id | BIGINT | -  | 产品 ID    | 外键，关联 products.id |

**reviews 表:**

| 字段名        | 数据类型      | 长度 | 说明      | 备注                |
|------------|-----------|----|---------|-------------------|
| id         | BIGINT    | -  | 评价主键 ID | 主键，非空，自动递增        |
| user_id    | BIGINT    | -  | 用户 ID   | 外键，关联 users.id    |
| product_id | BIGINT    | -  | 产品 ID   | 外键，关联 products.id |
| rating     | INT       | -  | 评分      | 非空，范围 1-5         |
| comment    | TEXT      | -  | 评论内容    | 可空                |
| created_at | TIMESTAMP | -  | 创建时间    | 默认值为当前时间          |

**admins 表:**

| 字段名      | 数据类型    | 长度 | 说明        | 备注         |
|----------|---------|----|-----------|------------|
| id       | BIGHT   | -  | 管理日志主键 ID | 主键，非空，自动递增 |
| username | VARCHAR | 50 | 管理员用户名    | 非空         |

|          |         |     |       |    |
|----------|---------|-----|-------|----|
| password | VARCHAR | 255 | 管理员密码 | 非空 |
|----------|---------|-----|-------|----|

## 2.6 系统出错处理设计

### 2.6.1 出错信息

| 错误类型 | 错误名字   | 输出信息       | 输出含义             | 处理方法                      |
|------|--------|------------|------------------|---------------------------|
| 网络错误 | 网络连接失败 | 网络连接失败     | 提示用户网络连接不可用      | 检查网络连接                    |
|      | 页面加载超时 | 页面加载超时     | 用户加载页面超时         | 刷新页面或检查网络连接               |
| 登陆错误 | 登录失败   | 密码错误       | 用户输入账号和密码对应失败    | 提示用户检查用户名和密码,或使用“忘记密码”功能。 |
|      | 身份验证失败 | 用户身份验证失败   | 用户输入邮箱或手机验证码有误   | 使用其他验证方式或者重试              |
| 权限错误 | 账户被封禁  | 账户已被封禁     | 该账户被封禁,无法使用      | 用户可联系管理员进行解封              |
|      | 用户权限不足 | 权限不足       | 用户进行不允许的操作       | 提示用户该操作需要管理员权限            |
| 支付错误 | 支付失败   | 支付失败       | 此处支付无效           | 重新支付或更换支付方式               |
| 库存错误 | 库存不足   | 该商品已售完     | 用户尝试购买或租借的商品已售完  | 提醒用户购买其他商品                |
| 上传错误 | 图片上传失败 | 上传失败       | 提醒商家该图片文件不可用     | 商家使用其他图片                  |
| 未知错误 | 未知错误   | 未知错误,请联系人员 | 未考虑到的错误,但是服务器会报错 | 用户反馈,开发人员迅速修正             |

### 2.6.2 补救措施

| 错误类型 | 错误名字 | 补救措施 |
|------|------|------|
|------|------|------|

|      |        |                                                                                                                                                 |
|------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 网络错误 | 网络连接失败 | <ol style="list-style-type: none"> <li>1. 用户检查设备的网络连接是否正常，尝试重启路由器或切换网络。</li> <li>2. 提供刷新页面的按钮，以便用户重新尝试。</li> </ol>                              |
|      | 页面加载超时 | <ol style="list-style-type: none"> <li>1. 提供页面重试按钮。</li> <li>2. 提示用户检查网络连接并刷新页面。</li> <li>3. 实现页面异步加载，以减轻加载时间压力。</li> </ol>                     |
| 登录错误 | 登录失败   | <ol style="list-style-type: none"> <li>1. 提供“忘记密码”功能，帮助用户重置密码。</li> <li>2. 提示用户检查输入的用户名和密码是否正确。</li> <li>3. 提供多次登录失败后的验证码验证以防止机器人攻击。</li> </ol> |
|      | 身份验证失败 | <ol style="list-style-type: none"> <li>1. 提示用户重新输入信息并检查是否有拼写错误。</li> <li>2. 在多次验证失败后，提供验证码验证以防止自动化攻击。</li> </ol>                                |
| 权限错误 | 账户被封禁  | <ol style="list-style-type: none"> <li>1. 提供联系客服入口，用户可以通过此入口申请解锁账户。</li> <li>2. 提供用户解禁申请流程，并说明禁用原因。</li> </ol>                                  |
|      | 用户权限不足 | <ol style="list-style-type: none"> <li>1. 提示用户该操作需要管理员权限。</li> <li>2. 提供联系管理员的快捷链接或按钮，便于用户申请权限。</li> </ol>                                      |
| 支付错误 | 支付失败   | <ol style="list-style-type: none"> <li>1. 提供用户选择其他支付方式的选项。</li> <li>3. 在支付失败时，允许用户保存订单并稍后重新支付。</li> </ol>                                       |
| 库存错误 | 库存不足   | <ol style="list-style-type: none"> <li>1. 提供用户选择其他商品或稍后购买的选项。</li> <li>2. 在商品页面显示库存实时更新。</li> <li>3. 提供“通知商品有货”功能，用户可以在有货时收到通知。</li> </ol>      |
| 上传错误 | 图片上传失败 | <ol style="list-style-type: none"> <li>1. 提供详细的错误提示信息，告诉用户失败的原因（文件大小、格式错误等）。</li> <li>2. 限制上传文件的大小和格式。</li> <li>3. 提供用户再次尝试上传的机会</li> </ol>     |
| 未知错误 | 未知错误   | <ol style="list-style-type: none"> <li>1. 提供用户反馈功能，让用户可以报告问题。</li> <li>2. 提供技术支持联系方式，以便用户获得帮助。</li> </ol>                                       |

### 2.6.3 系统维护设计

---

## • 错误处理与异常抛出

(1) **异常抛出** 在系统核心功能实现和重要的业务逻辑中，加入了错误捕捉和异常抛出的语句。例如，在用户登录、支付处理、数据存取等关键模块中，当检测到异常情况时，系统会抛出自定义的错误信息或异常对象，以便能够及时报告给运维人员。

(2) **错误日志** 通过在异常抛出时，系统会将错误信息写入日志文件中。这样，当系统运行中出现问题时，开发人员和运维人员可以查看错误日志来分析和定位问题。

## • 异常处理机制

(1) **用户提醒** 对于用户，系统会提供清晰的错误提示，说明出现问题的原因及可能的解决方法。例如，用户输入错误的账号和密码时，系统会返回“用户名或密码错误，请重试”信息。

(2) **用户反馈** 用户对于无法处理的错误，或者未知错误，平台会提供一个联系方式，用户可以反馈给客服。客服帮助解决异常，或者进一步联系开发人员对系统进行修正完善。